

1. Function *exist (const Point &p)*

En esta función lo que hacemos es determinar si un punto ya se encuentra almacenado dentro del Octree y lo implementé con una búsqueda recursiva. Lo primero que hago es verificar si está dentro de los límites del nodo actual que los defino con `bottomLeft`, si está fuera de los límites retorna *false*. Luego tengo mi caso base donde si el nodo actual es una hoja hago una búsqueda sobre el vector `points`, aquí si encuentra uno igual pues retorna *true*, sino retorna *false*. Finalmente uso la recursión para navegar entre los hijos.

```
bool Octree::exist(const Point &p) {
    if (p.x < bottomLeft.x || p.x >= bottomLeft.x + h || p.y < bottomLeft.y || p.y >= bottomLeft.y + h || p.z < bottomLeft.z || p.z >= bottomLeft.z + h) {
        return false;
    }

    if (isLeaf()) {
        for (const auto &pt : points) {
            if (pt == p) {
                return true;
            }
        }

        return false;
    }

    else {
        int oct = getOctant(p);
        return children[oct]→exist(p);
    }
}
```

2. Function *insert (const Point &p)*

Aquí lo que hacemos es añadir nuevos puntos al Octree con una subdivisión dinámica cuando alcanza la capacidad máxima, al igual que la función anterior, verifico si el punto pertenece a los límites y si no es así solo retorna sin cambios, si el nodo es una hoja y la cantidad de puntos es menor a la capacidad pues añadimos ese punto al vector `points` y se incrementa el contador, pero si el nodo es una hoja y alcanzó su capacidad pues subdividimos para crear otros 8 nuevos hijos con un tamaño $h/2$ y los redistribuimos los puntos del padre a las hojas que se crearon, y luego calculamos el octante para el nuevo punto `p`, pero si el nodo actual ya estuviera subdividido solo calculamos el octante correcto y se delega la insertación recursivamente a ese hijo.

```
void Octree::insert(const Point &p) {
    if (p.x < bottomLeft.x || p.x >= bottomLeft.x + h || p.y < bottomLeft.y || p.y >= bottomLeft.y + h || p.z < bottomLeft.z || p.z >= bottomLeft.z + h) {
        return;
    }

    if (isLeaf()) {
        if (points.size() < capacity) {
            points.push_back(p);
            nPoints++;
        }

        else {
            subdivide();
            int oct = getOctant(p);
            children[oct]→insert(p);
            nPoints++;
        }
    }

    else {
        int oct = getOctant(p);
        children[oct]→insert(p);
        nPoints++;
    }
}
```

3. Function *find_closest(const Point &p, int radius)*

Aquí hacemos la búsqueda del vecino más cercano dentro de un radio que tenemos como parámetro, lo primero que hago es definir un punto inicial que representa un estado nulo y luego calculo su distancia mínima al cuadrado para que las operaciones no sean costosas con las comparaciones, dentro tengo una función lambda para poder recorrer el Octree, aquí antes de explorar un nodo calculamos el centroide y la distancia al cuadrado desde este centro hasta el punto que tenemos, si su distancia es mucha lo descartamos, si llegamos a un nodo hoja iteramos sobre sus puntos, usamos la distancia euclidiana entre cada punto almacenado y el punto que tenemos, y si esa distancia es menor o igual a la distancia menor inicial actualizamos las variables. Por último, si al momento de recorrer todo el árbol no encontró ningún punto dentro del radio que le pasamos solo imprime NULL y devuelve un punto bandera inicializado en (-1, -1, -1).

```
Point Octree::find_closest(const Point &p, int radius) {
    Point closest(-1, -1, -1);
    double min_dist_sq = radius * radius;
    bool found = false;

    auto search = [&](auto &self, Octree *node) -> void {
        if (!node) {
            return;
        }

        double midX = node->bottomLeft.x + node->h / 2.0;
        double midY = node->bottomLeft.y + node->h / 2.0;
        double midZ = node->bottomLeft.z + node->h / 2.0;
        Point center(midX, midY, midZ);
        double distToCenterSq = distanceSq(p, center);
        double maxCuboRadiusSq = 3 * std::pow(node->h / 2.0, 2);

        if (distToCenterSq > min_dist_sq + maxCuboRadiusSq + 2 * std::sqrt(min_dist_sq * maxCuboRadiusSq)) {
            return;
        }

        if (node->isLeaf()) {
            for (const auto &pt : node->points){
                double d_sq = distanceSq(p, pt);
                if (d_sq <= min_dist_sq) {
                    min_dist_sq = d_sq;
                    closest = pt;
                    found = true;
                }
            }
        }

        else {
            for (int i = 0; i < 8; ++i) {
                self(self, node->children[i]);
            }
        }
    };

    search(search, this);

    if (!found) {
        std::cout << "NULL (No existe punto en el radio especificado)\n";
    }

    return closest;
}
```